

Slot Scheduling Engine

System Architecture & Algorithmic Workflow

1 Stage 1: Hard Constraints Filter (Eligibility Guard)

Before any slot is scored or ranked, it must pass 8 Hard Invariants configured in `backend/app/scheduling/constraints.json`. If a candidate slot fails even one check, it is discarded immediately:

- **ETA & Unload Duration Fit** (`arrival_before_slot_end`):

$$ETA + Unload\ Duration \leq Slot\ End\ Time$$

- **Facility Operating Hours** (`facility_operating_hours`): The slot window must fall inside facility working hours (e.g., 06:00 to 22:00 local time).
- **Dock Operational Status** (`dock_operational`): The dock must be ACTIVE (not closed, undergoing maintenance, or blocked by outage events).
- **Vehicle & Dock Physical Compatibility** (`dock_vehicle_compatibility`): Truck length, weight capacity (kg), clearance height, and door type must match.
- **Cargo & Temperature Compatibility** (`shipment_handling_compatibility`): Refrigerated / Reefer cargo requires cold-storage docks; hazmat or special equipment requires matching dock features.
- **Slot Capacity Availability** (`slot_capacity_available`): Remaining capacity must be > 0 after counting active appointments (PENDING_CONFIRMATION, CONFIRMED, IN_PROGRESS).
- **Single Active Appointment Rule** (`no_conflicting_active_appointment`): A shipment can only have 1 active appointment at a time.
- **Authoritative PostgreSQL ETA** (`latest_eta_only`): Evaluates the verified system-of-record ETA in PostgreSQL, ignoring stale client data.

2 Stage 2: Deterministic Weighted Multi-Factor Scoring

Eligible candidate slots are scored using a deterministic utility formula:

$$Score = S_{priority} + (w_{lateness} \times \Delta_{lateness}) + (w_{wait} \times \Delta_{wait}) + (w_{slack} \times \Delta_{slack}) - P_{dock}$$

Scoring Factors & Weights

Factor	Description	Weight / Impact
Base Priority ($S_{priority}$)	Priority code assigned to the shipment.	CRITICAL: +4,000 pts HIGH: +3,000 pts NORMAL: +2,000 pts LOW: +1,000 pts
Lateness Compensation ($w_{lateness}$)	Delay severity compared to original ETA.	+4 pts per minute of delay (capped at 720 min). Helps delayed loads recover lost time.
Driver Wait Time Penalty (w_{wait})	Idle time driver waits outside gate after arriving.	-6 pts per minute of wait time. Strongly favors earliest arrival fit to minimize driver idling.
Fit Slack Buffer (w_{slack})	Extra time buffer between unload completion and slot end.	+1 pt per minute of slack (capped at 120 min). Ensures operational cushion.
Dock Match Penalty (P_{dock})	Penalty for using a compatible fallback dock instead of an exact preferred dock type.	-25 pts penalty for non-exact dock match.

Deterministic Tie-Breaker

If two candidate slots produce the exact same score, ties are broken deterministically using `shipment_id` + `slot_id` (alphabetical sort). Zero randomness is allowed.

3 Stage 3: Transactional Concurrency (First-Come, First-Served)

When a driver selects a slot option:

- **Database-Level Unique Constraints:**

- `ux_active_appointment_per_slot` : Enforces at most 1 active appointment per slot window in PostgreSQL.
- `ux_current_active_appointment_per_shipment` : Enforces at most 1 active appointment per shipment.

- **Atomic Row-Locking (FOR UPDATE):**

When two drivers attempt to book the same slot at the exact same millisecond:

- **Driver 1 (Winner):** Obtains row lock, claims the slot, and receives status: `PENDING_CONFIRMATION`.
- **Driver 2 (Conflict):** Triggers PostgreSQL constraint violation (`IntegrityError`), receives `SLOT_CONFLICT_REFRESH_REQUIRED`, and automatically gets fresh replacement options without corrupting database state.



Summary

- **Priority:** Higher priority loads get base score boosts (`CRITICAL` = +4,000).
- **First-Come, First-Served:** Enforced atomically via PostgreSQL row locks and unique indexes.
- **Dock Compatibility:** Filtered as a hard constraint and penalized (-25 pts) if non-exact.
- **Driver Idling Prevention:** Heavily penalizes wait time (-6 pts/min) to get drivers unloaded as close to arrival as possible.